



Tecnicatura Universitaria en Inteligencia Artificial

Procesamiento del Lenguaje Natural

Informe Trabajo Práctico Final

Alumno: Jacob, Zahir - J-0625/4

Introducción

El objetivo de este trabajo fue hacer un asistente virtual para una empresa de electrodomésticos que pueda responder a las distintas consultas que pueda hacer el usuario usando información de varias fuentes. Se usó un modelo de lenguaje local para interpretar consultas, generar filtros y redactar respuestas a partir de la información recuperada.

Para resolver el trabajo se usaron 3 formas de almacenamiento distintas, por un lado una base vectorial para documentos no estructurados como las reseñas, manuales, tickets y las preguntas frecuentes. Por otro lado una base tabular para consultas que estuvieran relacionadas con productos, precios, stock y ventas. Y la última fue una base de grafos para representar las relaciones entre los productos, las categorías, subcategorías y marcas.

El trabajo está dividido en dos partes. En el primer ejercicio se implementó un sistema RAG con clasificación de intención, recuperación y generación de respuestas. Después ese sistema se extendió a un agente ReAct con herramientas que puede elegir que fuente consultar según la consulta del usuario.

Organización de las fuentes

El asistente tenía que responder preguntas variadas, algunas estaban relacionadas con textos, como las opiniones de usuarios, otras sobre precios o stock y también había consultas más orientadas a relaciones, como qué productos pertenecen a una categoría o que marcas hay de un producto. Cada tipo de dato tiene una forma distinta de consultarse, los textos largos se aprovechan mucho mejor con una base vectorial porque permite buscar fragmentos similares a una consulta del usuario. Los datos de productos y ventas funcionan mejor en las tabulares porque se pueden filtrar por campos como precio, stock o marca. Mientras que las relaciones entre los productos, categorías, subcategorías, se representan mejor en un grafo.

En resumen se utilizaron de la siguiente manera:

- **Base vectorial:** para información no estructurada, como reseñas, manuales, preguntas frecuentes y tickets de soporte. Permite responder consultas donde importa el significado del texto.
- **Base tabular:** para información estructurada, como productos, precios, stock, ventas, devoluciones e inventario. Permite aplicar filtros y realizar operaciones
- **Base de grafos:** para representar relaciones entre productos, categorías, subcategorías y marcas. Permite responder preguntas donde lo importante no es solo un dato sino cómo se conectan los elementos

Esta separación permite que el sistema elija la fuente más adecuada según la intención de la consulta.

Diseño general del trabajo

El trabajo se organizó como un sistema de recuperación y respuesta dividido en varias etapas. Primero se cargaron y se prepararon las fuentes de información del dataset y según el tipo de dato, la información fue derivada a la base correspondiente. El flujo general del sistema comienza con la consulta del usuario. Esa consulta pasa por un clasificador de intención que decide qué fuente conviene consultar. Si la pregunta está relacionada con documentos, reseñas, manuales o tickets, se usa la recuperación vectorial. Si la pregunta requiere filtrar productos por precio, stock, marca o características, se consulta la fuente tabular. Si la pregunta está orientada a relaciones entre productos, categorías o marcas, se usa la base de grafos. Una vez que se selecciona la fuente se recupera la información necesaria. En la parte vectorial se combinó búsqueda semántica, BM25 y re-rank. En la parte tabular el modelo local genera filtros que luego se aplican sobre los DataFrames. En la parte de grafos el modelo local transforma la consulta del usuario en una consulta Cypher para GrafitoDB.

En la parte final con los resultados recuperados, el sistema genera una respuesta para el usuario. En las consultas textuales se usa el modelo de lenguaje local para redactar a partir del contexto. En cambio cuando se trabaja con datos tabulares o de grafos, se devuelven directamente los resultados obtenidos, para no cometer errores en precios, stock o relaciones.

Ejercicio 1: RAG

Base de datos vectorial

La información guardada en esta base no se consulta bien con filtros exactos porque el usuario puede preguntar con palabras diferentes a las que aparecen en los documentos así que se usó una búsqueda semántica.

Para representar los textos se usó el modelo de embeddings “intfloat/multilingual-e5-small”. Se eligió este modelo porque es multilingüe así que puede trabajar con consultas en español y otros idiomas y además es liviano.

```
# Modelo multilingüe apto para español.  
# Es chico, rápido y sirve para búsqueda semántica.  
embedding_model_name = "intfloat/multilingual-e5-small"  
  
embedding_model = SentenceTransformer(embedding_model_name)
```

Antes de cargar los textos en ChromaDB los documentos se dividieron en fragmentos usando “MarkdownHeaderTextSplitter” para los manuales, y para el resto de los textos se usó un splitter recursivo donde se configuró un tamaño de fragmento de 700 caracteres y un solapamiento de 100 caracteres. La idea de esta división fue evitar guardar textos demasiado largos como una sola unidad porque puede hacer que la recuperación sea mucho menos precisa. El solapamiento hace que no se pierda contexto entre un fragmento y el que sigue.

```
markdown_splitter = MarkdownHeaderTextSplitter(  
    headers_to_split_on=headers_to_split_on,  
    strip_headers=False  
)  
  
text_splitter = RecursiveCharacterTextSplitter(  
    chunk_size=700,  
    chunk_overlap=100,  
    separators=["\n\n", "\n", ". ", " ", ""]  
)
```

Después, cada fragmento fue convertido a embedding y cargado en una colección de ChromaDB junto con su metadata. Esa metadata permite saber de dónde viene cada fragmento, por ejemplo si pertenece a una reseña, un manual, un ticket o una FAQ.

Recuperación vectorial, búsqueda híbrida y re-rank

Para consultar la base vectorial se armó una función llamada “`buscar_en_chroma()`” que recibe una consulta, la cantidad de resultados y filtros opcionales. Lo que hace es generar el embedding de la consulta y usarlo para buscar en ChromaDB los fragmentos más parecidos. Además se armó una recuperación híbrida. La búsqueda semántica sirve para encontrar textos con significado similar aunque no sean las mismas palabras, pero a veces también conviene buscar coincidencias más directas así que se sumó BM25 para usar la frecuencia de palabras.

La función “`recuperar_textos()`” junta todo eso, primero obtiene los candidatos usando tanto la búsqueda semántica como BM25, después combina los resultados y les aplica un re-rank, este re-rank vuelve a ordenar los fragmentos según qué tan parecidos son a la consulta para quedarse con los más relevantes.

Base tabular y generación de filtros

Para las consultas tabulares los archivos del dataset fueron cargados en DataFrames con Pandas para poder trabajar con columnas y poder aplicarles filtros. No se le pasa todo el dataset al modelo sino que se le armó un resumen con información como categorías, marcas, subcategorías, colores, voltajes y rangos de precios. Este resumen le da al modelo una idea general de qué tipo de datos existen permitiéndole interpretar mejor la consulta sin necesidad de acceder a todas las filas del dataset. La consulta la interpreta el modelo local y devuelve un diccionario de filtros en formato JSON.

La función “`buscar_productos_tabular()`” recibe los filtros y devuelve los productos que cumplen las condiciones, y a partir de esa función se armó

“buscar_productos_natural()” que primero genera los filtros usando el modelo y después hace la búsqueda. Separar estos pasos ayuda a que el modelo solo se encargue de interpretar la consulta mientras que los resultados salen directamente de los datos y se evita que aparezcan productos o valores inventados.

```
def buscar_productos_natural(consulta_usuario, top_k=10):  
    # Primero genera filtros con el LLM local.  
    # Después usa esos filtros en la búsqueda tabular.  
  
    filtros = generar_filtros_tabulares_llm(consulta_usuario)  
  
    resultado = buscar_productos_tabular(  
        filtros=filtros,  
        top_k=top_k  
    )  
  
    return filtros, resultado
```

Base de grafos y generación consultas Cypher

Para representar las relaciones del catálogo se usó una base de grafos local con GrafitoDB, conectando productos con sus categorías, subcategorías y marcas. Las relaciones se generaron a partir de un DataFrame y se cargaron en GrafitoDB usando Cypher para las consultas. Para simplificar el uso se implementó una función que convierte preguntas en lenguaje natural a consultas Cypher mediante el modelo local y luego las ejecuta en el grafo. Así el modelo solo traduce la intención del usuario mientras que la información se obtiene directamente de la base de datos, lo que aumenta la precisión de las respuestas.

```
Consulta generada: MATCH (p:Producto)-[:PERTENECE_A]->(c:Categoria {nombre: 'Cocina'})  
RETURN p.nombre, c.nombre
```

Clasificador de intención

Para decidir que fuente de datos se tenía que consultar en cada caso se usó un clasificador de intención porque el sistema no tiene que tratar todas las preguntas de la misma manera. Se probaron dos enfoques, el primero fue un clasificador supervisado entrenado con preguntas sintéticas y se usó un pipeline con TF-IDF y

Regresión logística para convertir el texto en valores numericos y separar las clases segun esos valores. El segundo enfoque fue un clasificador basado en LLM con few-shot prompting. Para este se construye un prompt que incluye varios ejemplos de preguntas junto con su categoría correspondiente (por ejemplo, vectorial, tabular o grafo) y cuando llega una consulta nueva, se la agrega al mismo prompt y el modelo decide a qué categoría pertenece tomando como referencia esos ejemplos. Ambos enfoques se compararon usando métricas de clasificación. En las pruebas realizadas los dos tuvieron buenos resultados pero el clasificador basado en LLM fue elegido para el pipeline final porque resultó más flexible ante consultas escritas de distintas formas.

Clasificador supervisado:

```
...      precision    recall  f1-score   support

      grafo         1.00      1.00      1.00         3
      tabular        1.00      1.00      1.00         4
      vectorial       1.00      1.00      1.00         4

 accuracy          1.00          1.00          1.00        11
 macro avg         1.00      1.00      1.00        11
 weighted avg      1.00      1.00      1.00        11

Matriz de confusión:
[[4 0 0]
 [0 4 0]
 [0 0 3]]
```

LLM con few-shot prompting:

```
... Evaluación del clasificador LLM few-shot:
      precision    recall  f1-score   support

      grafo         1.00      1.00      1.00         3
      tabular        1.00      1.00      1.00         4
      vectorial       1.00      1.00      1.00         4

 accuracy          1.00          1.00          1.00        11
 macro avg         1.00      1.00      1.00        11
 weighted avg      1.00      1.00      1.00        11

Matriz de confusión LLM:
[[4 0 0]
 [0 4 0]
 [0 0 3]]
```

Pipeline de recuperación y checkpoint

Después de implementar las tres fuentes de datos y el clasificador de intención, se integró todo en un único pipeline de recuperación. La función principal recibe la consulta del usuario, detecta la intención y deriva la búsqueda a la fuente correspondiente. En esta etapa no se genera todavía una respuesta final, sino que se controla si la recuperación está funcionando correctamente. Para eso se armó un checkpoint con cinco prompts. En cada caso se muestra la consulta original, la intención detectada, la fuente seleccionada, la consulta generada cuando corresponde y los resultados recuperados. Esto permitió verificar que las preguntas sobre opiniones fueran a la base vectorial, que las consultas sobre precio o stock fueran a la fuente tabular y que las preguntas sobre relaciones generaran una consulta Cypher para el grafo. En las capturas se puede ver un ejemplo de esta validación, donde el sistema muestra no solo la respuesta de recuperación, sino también información como metadata, fuente y puntajes, lo que permite revisar si los documentos o registros recuperados son coherentes con la consulta del usuario.

Salidas Checkpoint

```
=====
PROMPT 1
=====
Consulta: ¿Cómo uso mi licuadora para hacer smoothies?
Intención detectada: vectorial
Fuente seleccionada: ChromaDB + BM25 + ReRank
Consulta generada: None

Resultados recuperados:

Resultado 1
ID: manual_manual_P0004_Compacto_Licuadora_md_8_chunk_0
Fuente: manuales_productos
Tipo: manual
Score rerank: 0.8997
Distancia semántica: 0.2006242722272873
Score BM25: 21.196226119995117
Texto: Manual Técnico - Compacto Licuadora Procedimientos de Uso PROCEDIMIENTO 1: Preparar Smoothie de Frutas
## Procedimientos de Uso
### PROCEDIMIENTO 1: Preparar Smoothie de Frutas
**Dificultad:** Fácil | **Tiempo:** 3-5 minutos
**Pasos:**
1. Lavar y cortar las frutas en trozos medianos (2-3 cm)
2. Colocar los ingredientes líquidos primero (leche, yogurt, jugo)
3. Agregar las frutas y hielo en la jarra
4. Cerrar la tapa asegurándose del cierre hermético
5. Comenzar en velocidad baja (nivel 1-2) por 10 segundos
6. Aumentar gradualmente a velocidad alta (nivel 4-5)
7. Procesar durante 45-60s

=====
PROMPT 2
=====
No se encontraron productos con esos filtros.
Consulta: ¿Cuáles son las licuadoras de menos de 200 dólares?
Intención detectada: tabular
Fuente seleccionada: Pandas
Consulta generada: {'nombre_contiene': 'licuadora', 'precio_max': 200.0, 'ordenar_por': 'precio_usd', 'ascendente': True}

Resultados recuperados:

  id_producto  nombre  categoria  subcategoria  marca  precio_usd  stock  voltaje  garantia_meses
=====
```

```
=====
PROMPT 3
=====
Consulta: ¿Qué opinan los usuarios de esta cafetera?
Intención detectada: vectorial
Fuente seleccionada: ChromaDB + BM25 + ReRank
Consulta generada: None

Resultados recuperados:

Resultado 1
ID: resena_resena_R04450_chunk_0
Fuente: resenas_usuarios
Tipo: resena
Score rerank: 0.8662
Distancia semántica: 0.26765725016593933
Score BM25: None
Texto: Fecha: 2025-07-30
Usuario: [oculto]
Teléfono: [oculto]
Producto: Cafetera (P0122)
Puntaje: 3/5
Provincia: Santa Fe

Les cuento... Tiene sus pros y contras con Cafetera. Tiene cosas buenas y cosas malas. Por el precio está bien, pero no esperen maravillas. Espero les sirva!
```

```
=====
PROMPT 4
=====
Consulta: Quiero una licuadora con buenas reseñas
Intención detectada: vectorial
Fuente seleccionada: ChromaDB + BM25 + ReRank
Consulta generada: None

Resultados recuperados:

Resultado 1
ID: resena_resena_R04054_chunk_0
Fuente: resenas_usuarios
Tipo: resena
Score rerank: 0.8877
Distancia semántica: 0.22459633648395538
Score BM25: None
Texto: Fecha: 2024-11-09
Usuario: [oculto]
Teléfono: [oculto]
Producto: Licuadora (P0002)
Puntaje: 4/5
Provincia: Formosa

Les cuento... Mejor de lo que esperaba con Licuadora. Lo compré hace varios meses y es muy práctico, además de robusto. También destaco que es preciso. La relación calidad-precio es excelente. Gracias por leer.
```

```
=====
PROMPT 5
=====
Consulta: ¿Qué productos están relacionados con la categoría Cocina?
Intención detectada: grafo
Fuente seleccionada: GrafitoDB
Consulta generada: MATCH (p:Producto)-[:PERTENECE_A]->(c:Categoria {nombre: 'Cocina'})
RETURN p.nombre, c.nombre

Resultados recuperados:
Resultado 1: {'p.nombre': 'Licuadora', 'c.nombre': 'Cocina'}
Resultado 2: {'p.nombre': 'Licuadora', 'c.nombre': 'Cocina'}
Resultado 3: {'p.nombre': 'Plus Licuadora Pro', 'c.nombre': 'Cocina'}
```

Generación final y memoria conversacional

Después del checkpoint de recuperación se agregó la generación final de respuestas. En esta etapa el sistema ya no muestra solamente fragmentos o registros recuperados, sino que construye una respuesta para el usuario. La generación se resolvió de distinta manera según la fuente consultada. Para las consultas vectoriales, como reseñas, manuales o tickets, se usó el modelo de lenguaje local para redactar una respuesta a partir del contexto recuperado. En cambio, para las consultas tabulares y de grafos, preferí armar la respuesta con los resultados recuperados directamente. De esta forma se evita que el modelo agregue productos, precios, stock o relaciones que no salieron de la base. También se agregó una memoria conversacional básica. Esta memoria guarda los turnos de conversación en una lista, incluyendo la consulta del usuario, la intención detectada, la fuente utilizada y la respuesta generada. No se guarda todo el contexto recuperado, ya que eso haría crecer demasiado el historial. El contexto recuperado se usa en el turno actual, mientras que la memoria conserva una versión más liviana de la conversación.

Salidas finales:

```
=====
Consulta: ¿Cómo uso mi licuadora para hacer smoothies?
```

```
Intención: vectorial
```

```
Fuente: ChromaDB + BM25 + ReRank
```

```
Respuesta final:
```

```
Para preparar un smoothie con tu licuadora, sigue estos pasos:
```

1. Lavar y cortar las frutas en trozos medianos (2-3 cm).
2. Coloca los ingredientes líquidos primero (leche, yogurt, jugo).
3. Agrega las frutas y hielo en la jarra.
4. Cierra la tapa asegurándote de que esté cerrada herméticamente.
5. Comienza en velocidad baja (nivel 1-2) por 10 segundos.
6. Aumenta gradualmente a velocidad alta (nivel 4-5).
7. Procesa durante 45-60 segundos hasta que el smoothie tenga textura homogénea.
8. Utiliza la función PULSE para romper trozos grandes si es necesario.

```
Siguiendo estos pasos, podrás preparar tu smoothie de manera efectiva con tu licuadora.
```

```
=====
Consulta: ¿Cuáles son las licuadoras de menos de 200 dólares?
No se encontraron productos con esos filtros.
Intención: tabular
Fuente: Pandas

Respuesta final:
No se encontraron productos que cumplan con esos filtros. Probá reformular la consulta con más detalle.
```

```
=====
Consulta: ¿Qué opinan los usuarios de esta cafetera?
Intención: vectorial
Fuente: ChromaDB + BM25 + ReRank

Respuesta final:
Según las reseñas recuperadas, los usuarios tienen opiniones variadas sobre la cafetera. Algunos valoran su relación calidad-precio, como la reseña de Río Negro que menciona que es muy duradero y tiene una excelente relación calidad-precio. Otros destacan su versatilidad y facilidad de uso, como la reseña de Philips que menciona que es muy fácil de usar y limpiar. Sin embargo, también existen críticas negativas. Una reseña de Santa Cruz menciona que es muy caro y inestable, y tuvo problemas con el servicio técnico. Otra reseña de Catamarca señala que es muy se calienta y consume mucho energía, lo que la califica con un pésimo puntaje de 2/5. Además, se menciona un problema común de sobrecalentamiento en un ticket de soporte, lo cual afecta la satisfacción del usuario. Este problema se clasificó como alta severidad y está relacionado con la categoría eléctrica.
```

```
=====
Consulta: Quiero una licuadora con buenas reseñas
Intención: vectorial
Fuente: ChromaDB + BM25 + ReRank

Respuesta final:
Según las reseñas recuperadas, hay opiniones mixtas sobre la licuadora. Algunos usuarios recomiendan la licuadora por su relación calidad-precio, su facilidad de uso, su compactura y su robustez. Por ejemplo, dos usuarios destacaron que es muy confiable y práctico, y que tiene una relación calidad-precio muy buena. Sin embargo, otros usuarios mencionaron que es un poco ruidosa y que el motor se calienta un poco al usarla por tiempos prolongados.
```

```
=====
Consulta: ¿Qué productos están relacionados con la categoría Cocina?
Intención: grafo
Fuente: GrafitoDB

Respuesta final:
Los productos únicos recuperados desde el grafo son:
- Licuadora
- Plus Licuadora Pro
- Compacto Licuadora
- Ultra Licuadora
- Procesadora
- Deluxe Procesadora
```

Ejercicio 2: Agente ReAct con herramientas

En este ejercicio se tomó al sistema RAG del punto anterior y se adaptó para que funcione como un agente con el enfoque ReAct. La diferencia principal es que en vez de responder siempre de forma directa, el agente primero analiza la consulta y decide qué herramienta conviene usar. Para esto se definieron cuatro herramientas. “doc_search” se encarga de buscar información en documentos, como manuales, reseñas, preguntas frecuentes y tickets de soporte. “table_search” se usa para consultas sobre productos, precios, stock o características concretas. “graph_search” permite consultar relaciones entre productos, categorías, subcategorías y marcas dentro de la base de grafos. Por último, “analytics_tool” permite hacer análisis sobre datos tabulares usando SQL y también generar gráficos

con matplotlib. Las herramientas se definieron con “LangChain” usando el decorador “@tool”. Esto permite tomar funciones comunes de Python y dejarlas disponibles para que el agente pueda ejecutarlas cuando las necesite. La decisión de qué herramienta usar la toma el modelo local a partir de la consulta del usuario. El funcionamiento general sigue la lógica ReAct, primero el modelo genera un razonamiento breve sobre qué necesita hacer, después elige una acción, ejecuta la herramienta correspondiente, observa el resultado y finalmente redacta una respuesta para el usuario.

En las pruebas del agente se imprimieron las distintas partes de la ejecución para poder revisar qué decisión estaba tomando el sistema. En la salida se muestra la consulta original, el pensamiento generado por el modelo, la acción elegida, la entrada enviada a la herramienta, la observación devuelta y la respuesta final. Esto sirvió para controlar que el agente no eligiera herramientas al azar, or ejemplo en una consulta de análisis de datos debía seleccionar `analytics\_tool`, mientras que en una consulta sobre documentos debía usar `doc\_search`.

```
Consulta: productos más vendidos
Thought: El usuario está interesado en conocer los productos más vendidos, lo que se puede obtener mejor a través de un análisis de datos.
Action: analytics_tool
Action input: productos más vendidos
```

```
Consulta: ¿Qué opinan los usuarios de esta cafetera?
Thought: La consulta busca opiniones de usuarios sobre una cafetera específica. La mejor herramienta para obtener esta información sería doc_search.
Action: doc_search
Action input: opiniones de usuarios sobre esta cafetera
```

La herramienta “analytics_tool” se usó para las consultas donde había que hacer algún cálculo o resumen con los datos, por ejemplo cuáles fueron los más vendidos o cuál es el stock promedio por categoría. Para resolver esas consultas se pasaron los DataFrames a una base SQLite en memoria y se hicieron consultas SQL sobre esas tablas. Después cuando el resultado quedaba más claro de forma visual, se generó un gráfico con matplotlib. Esto permite que el agente responda consultas más analíticas usando los datos cargados en vez de depender solo del texto recuperado o de una respuesta generada por el modelo.

Resultados de pruebas

Se probaron consultas sobre uso de productos, opiniones de usuarios, filtros de precio y stock, y relaciones entre productos y categorías. En general el sistema logró derivar correctamente las preguntas a la base vectorial, tabular o de grafos según correspondía. Por ejemplo, las preguntas sobre manuales y reseñas fueron a la recuperación vectorial, las preguntas sobre precios o stock fueron a la fuente tabular, y las preguntas sobre relaciones fueron convertidas en consultas Cypher para GrafitoDB. También se realizaron pruebas con el agente ReAct. En este caso se revisó que el modelo eligiera la herramienta adecuada antes de responder. Las consultas sobre documentos fueron resueltas con `doc_search`, las consultas sobre productos con `table_search`, las consultas sobre relaciones con `graph_search` y las consultas de análisis con `analytics_tool`.

```
Usuario: ¿Qué opinan los usuarios de esta cafetera?
Asistente: Hay opiniones positivas y negativas sobre esta cafetera. Algunos usuarios recomiendan su relación calidad-precio y durabilidad, mientras que otros se

Usuario: licuadoras de menos de 400 dólares con stock
Asistente: Encontré tres licuadoras que cumplen con tus requisitos:

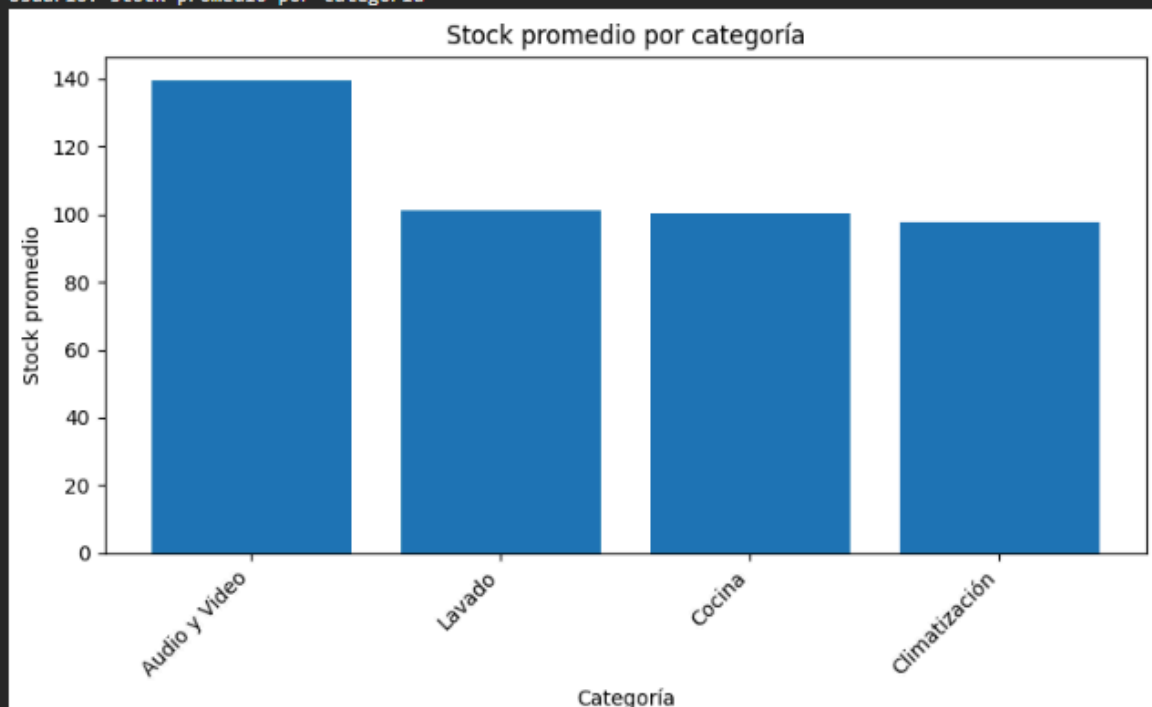
1. La Licuadora de TechHome cuesta $283.63 y está en stock.
2. La Plus Licuadora Pro de TechHome cuesta $329.07 y también está disponible.
3. La Compacto Licuadora de ChefMaster cuesta $259.42 y está en stock.

Espero que encuentres lo que buscas.

Usuario: productos relacionados con la categoría Cocina
Asistente: Los productos relacionados con la categoría Cocina son:

- Licuadora
- Plus Licuadora Pro
- Compacto Licuadora
- Ultra Licuadora
- Procesadora
- Deluxe Procesadora
```

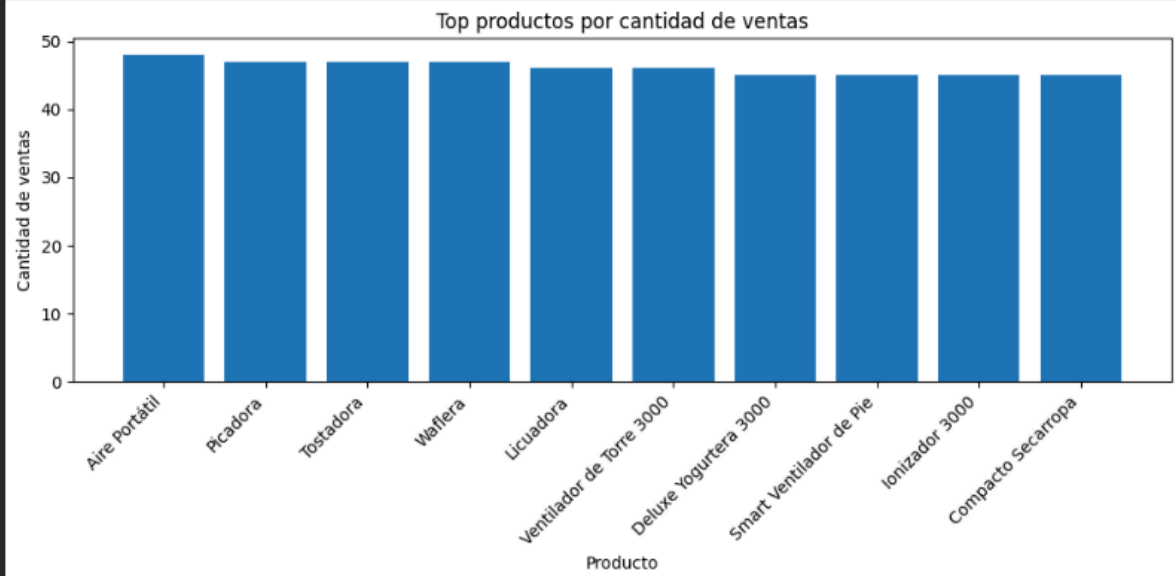
Usuario: stock promedio por categoría



Asistente: El stock promedio por categoría es como sigue:

- Audio y Video: 139.5
- Lavado: 101.19
- Cocina: 100.12
- Climatización: 97.6

Usuario: productos más vendidos



Asistente: Los productos más vendidos son:

1. Aire Portátil (id: P0154) - Climatización - ClimaTech - 48 ventas
2. Picadora (id: P0015) - Cocina - HomeChef - 47 ventas
3. Tostadora (id: P0056) - Cocina - CookElite - 47 ventas
4. Waflera (id: P0070) - Cocina - CookElite - 47 ventas
5. Licuadora (id: P0006) - Cocina - ChefMaster - 46 ventas

Estos son los cinco productos con mayor cantidad de ventas según nuestros datos.

Usuario: como uso mi licuadora para hacer smoothies?

Asistente: Para preparar un smoothie con tu licuadora, sigue estos pasos:

1. Lava y corta las frutas en trozos medianos.
2. Coloca los ingredientes líquidos primero (leche, yogurt, jugo).
3. Agrega las frutas y hielo en la jarra.
4. Cierra la tapa asegurándote de que esté cerrada herméticamente.
5. Comienza en velocidad baja (nivel 1-2) por 10 segundos.
6. Aumenta gradualmente a velocidad alta (nivel 4-5).
7. Procesa durante 45-60 segundos hasta que el smoothie tenga textura homogénea.
8. Usa la función PULSE si hay trozos grandes que no se rompen.

Recuerda servir el smoothie inmediatamente para obtener la mejor textura y sabor.

Usuario: salir

Fin del chat.

Limitaciones y mejoras posibles

Durante las pruebas el sistema logró responder correctamente varias consultas, pero también aparecieron algunas limitaciones. Una de ellas es que la memoria conversacional implementada es básica. El sistema guarda los turnos anteriores, pero no siempre interpreta bien consultas incompletas que dependen mucho del mensaje anterior. Otra limitación es que el agente ReAct elige una herramienta principal por consulta, esto funciona bien para la mayoría de los casos probados, pero podría quedarse corto en preguntas que necesiten combinar varias fuentes al mismo tiempo, por ejemplo una consulta que mezcle opiniones de usuarios con filtros de precio o stock. También hay que tener en cuenta que se usó un modelo de lenguaje local, esto permite trabajar sin depender de APIs externas, pero al mismo tiempo puede hacer que algunas respuestas sean menos precisas o más simples que las que generaría un modelo más grande. Por eso fue importante limitar el rol del modelo, en varios casos solo interpreta la consulta o genera filtros, mientras que los datos finales salen directamente de las fuentes cargadas.

Como mejora futura se podría agregar más variedad de consultas dentro de la herramienta “analytics_tool”. Actualmente resuelve algunos casos definidos, como productos más vendidos, devoluciones, tickets o stock promedio, pero se podrían sumar otros análisis simples, por ejemplo ventas por categoría, productos con menor stock o comparación entre marcas.

Conclusión

En el caso del RAG, lo más importante fue la etapa de recuperación. Si la información recuperada era correcta, la respuesta final tenía muchas más probabilidades de ser útil. Por eso fue necesario separar bien las fuentes y usar un clasificador de intención que decidiera si la consulta debía ir a la base vectorial, a las tablas o al grafo. De esta manera cada tipo de pregunta se resolvía con la fuente más adecuada. El agente ReAct agregó una capa más de decisión. A diferencia del RAG inicial, el agente no sigue siempre el mismo recorrido sino que primero analiza la consulta y elige una herramienta, lo que permitió trabajar de forma más flexible con búsquedas en documentos, filtros tabulares, consultas al grafo y análisis con SQL. Además al mostrar el pensamiento, la acción, la observación y la respuesta final, se pudo revisar mejor qué estaba haciendo el sistema en cada prueba.

Como cierre, el RAG resultó más directo y ordenado para recuperar información y responder, mientras que ReAct fue más útil para organizar varias herramientas dentro de un mismo asistente. La principal limitación es que ambos enfoques dependen mucho de que la recuperación, los filtros o la herramienta elegida sean correctos. Si esa parte falla, la respuesta final también se ve afectada.